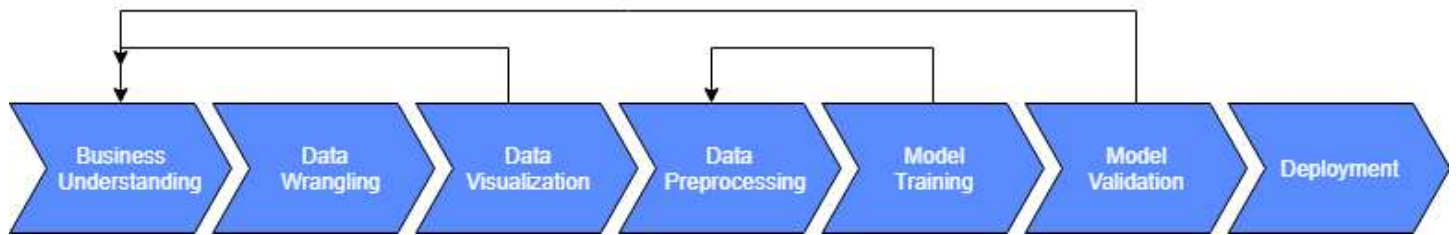


PreProcessing using scikit-learn



Agenda

1. Introduction to Preprocessing
2. StandardScaler
3. MinMaxScaler
4. RobustScaler
5. Normalization
6. Binarization
7. Encoding Categorical (Ordinal & Nominal) Features
8. Imputation
9. Polynomial Features
10. Custom Transformer
11. HashingVectorizer
12. Image using skimage

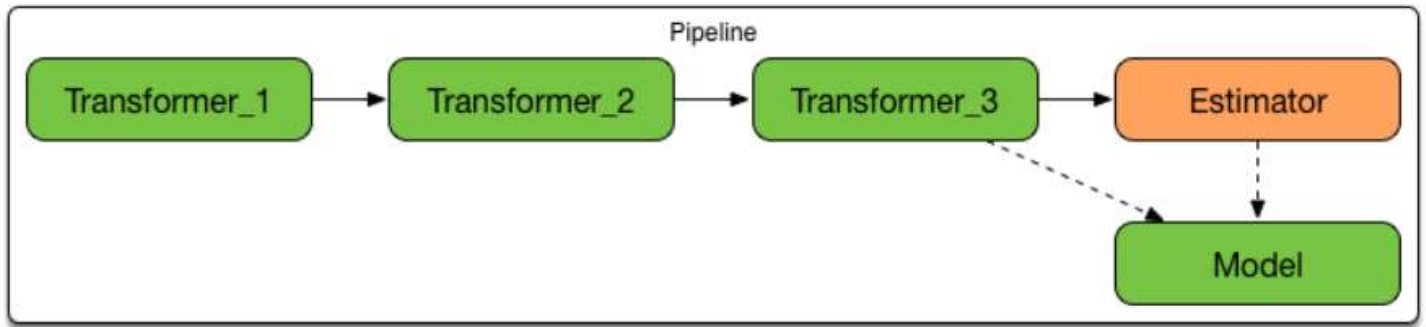
Common import

```
import numpy as np
import pandas as pd
import seaborn as sns; sns.set(color_codes=True)
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt
%matplotlib inline
```

1. Introduction to PreProcessing

- Learning algorithms have affinity towards certain pattern of data.
- Unscaled or unstandardized data have might have unacceptable prediction

- Learning algorithms understands only number, converting text, image to number is required
- Preprocessing refers to transformation before feeding to machine learning



2. StandardScaler

- The StandardScaler assumes your data is normally distributed within each feature and will scale them such that the distribution is now centred around 0, with a standard deviation of 1.
- Calculate - Subtract mean of column & div by standard deviation

$$\frac{x_i - \text{mean}(x)}{\text{stdev}(x)}$$

- If data is not normally distributed, this is not the best scaler to use.

```

#Generating normally distributed data

df = pd.DataFrame({
    'x1': np.random.normal(0, 1, 10000),
    'x2': np.random.normal(10, 5, 10000),
    'x3': np.random.normal(50, 10, 10000)
})

```

df

	x1	x2	x3
0	0.463124	2.365363	40.370022
1	0.122437	16.457868	45.709240
2	-1.015430	12.098995	61.900859
3	-0.221987	10.822430	35.914164
4	0.187119	1.450121	49.344819
...	...	...	...
9995	-0.358891	14.150942	52.192122
9996	0.423233	10.966956	58.601257
9997	1.174358	7.913014	43.470246
9998	-0.513935	8.299045	39.401472
9999	0.124785	9.065921	45.239020

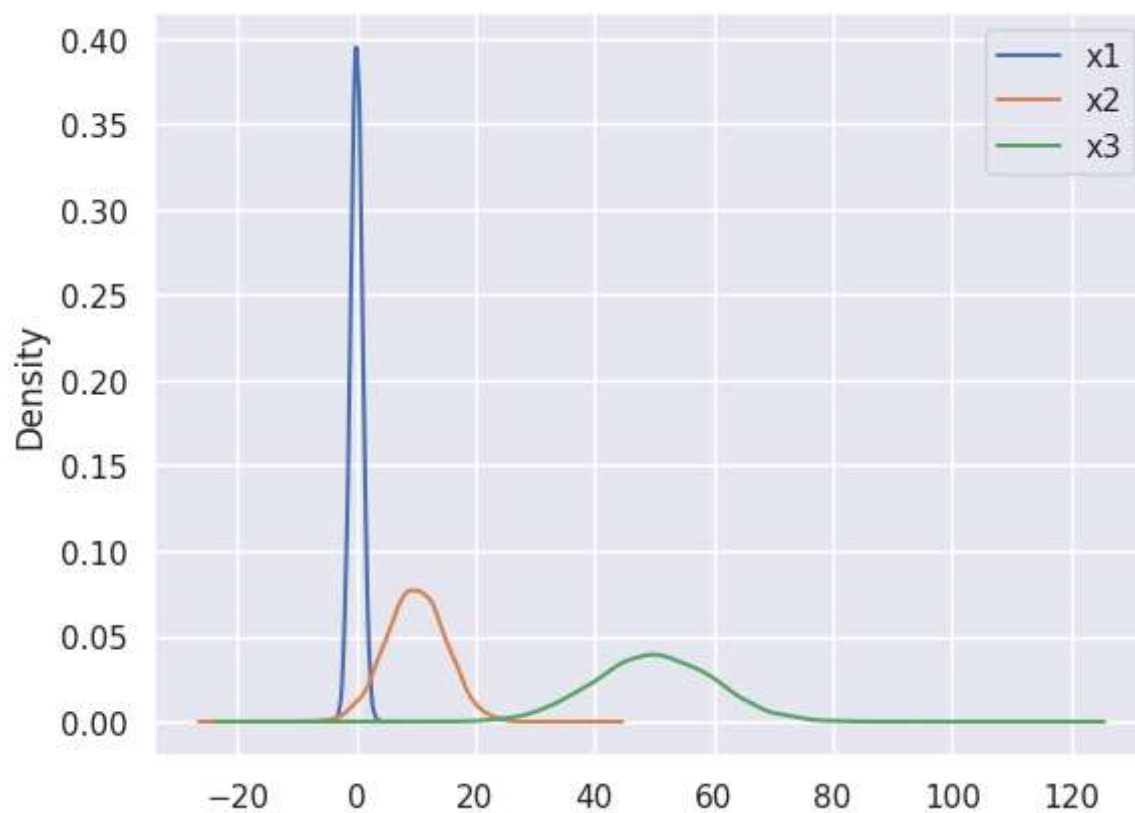
df.describe()

	x1	x2	x3
count	10000.000000	10000.000000	10000.000000
mean	-0.004755	9.976345	49.984226
std	0.996212	4.971475	9.993464
min	-3.600523	-8.676614	13.567828
25%	-0.666085	6.607681	43.277881
50%	0.000930	9.972108	49.988434
75%	0.661591	13.360785	56.922647
max	3.770515	26.871835	88.228439

```
# plotting data
```

```
df.plot.kde()
```

<Axes: ylabel='Density'>



```
from sklearn.preprocessing import StandardScaler  
standardscaler = StandardScaler()
```

```
data_tf = standardscaler.fit_transform(df)
```

```
type(data_tf)
```

numpy.ndarray

```
df = pd.DataFrame(data_tf, columns=['x1', 'x2', 'x3'])
```

df

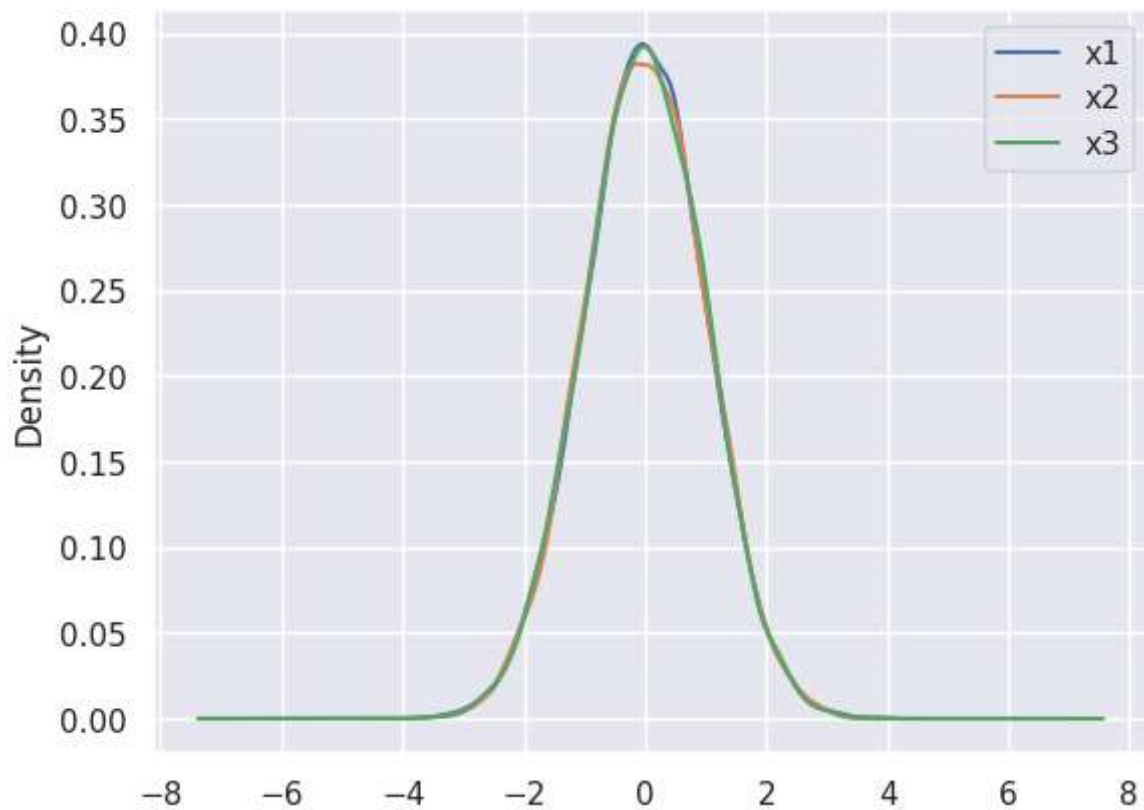
	x1	x2	x3
0	0.469682	-1.531007	-0.962097
1	0.127682	1.303808	-0.427800
2	-1.014569	0.426987	1.192502
3	-0.218069	0.170196	-1.407997
4	0.192613	-1.715115	-0.063986
...	...	...	...
9995	-0.355500	0.839752	0.220945
9996	0.429637	0.199269	0.862310
9997	1.183655	-0.415055	-0.651857
9998	-0.511142	-0.337402	-1.059020
9999	0.130039	-0.183139	-0.474855

df.describe()

	x1	x2	x3
count	1.000000e+04	1.000000e+04	1.000000e+04
mean	3.552714e-18	1.101341e-16	2.987832e-16
std	1.000050e+00	1.000050e+00	1.000050e+00
min	-3.609620e+00	-3.752185e+00	-3.644204e+00
25%	-6.638779e-01	-6.776325e-01	-6.711066e-01
50%	5.706764e-03	-8.522445e-04	4.211805e-04
75%	6.689133e-01	6.808058e-01	6.943306e-01
max	3.789813e+00	3.398657e+00	3.827114e+00

```
df.plot.kde()
```

<Axes: ylabel='Density'>



3. MinMaxScaler

- One of the most popular
- Calculate - Subtract min of column & div by difference between max & min

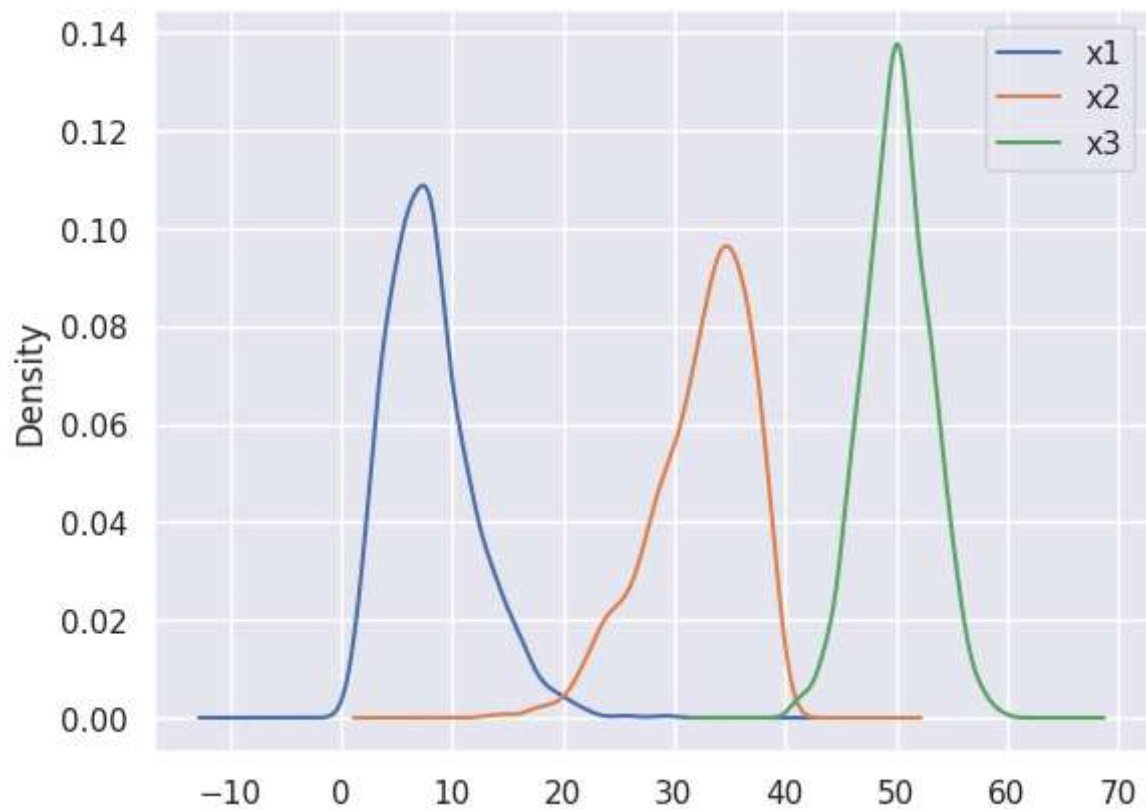
$$\frac{x_i - \min(x)}{\max(x) - \min(x)}$$

- Data shifts between 0 & 1
- If distribution not suitable for StandardScaler, this scaler works out.
- Sensitive to outliers

```
df = pd.DataFrame({  
    # positive skew  
    'x1': np.random.chisquare(8, 1000),  
    # negative skew  
    'x2': np.random.beta(8, 2, 1000) * 40,  
    # no skew  
    'x3': np.random.normal(50, 3, 1000)  
})
```

```
df.plot.kde()
```

<Axes: ylabel='Density'>

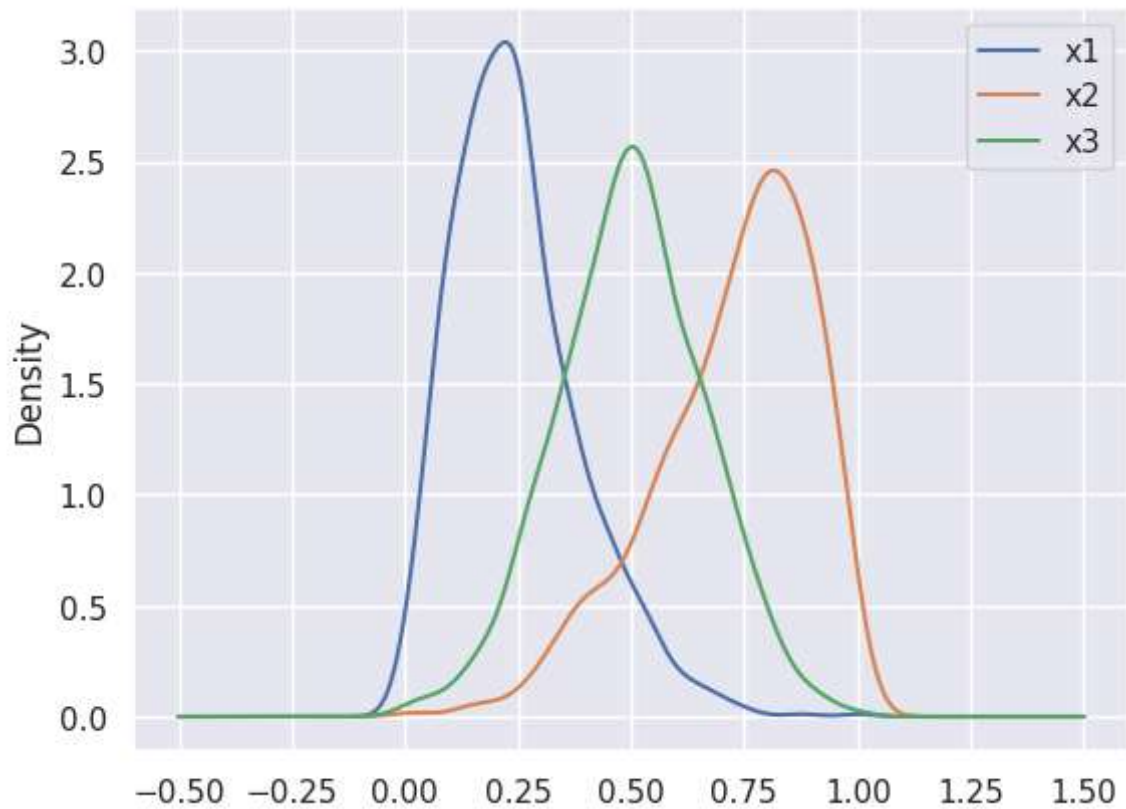


```
from sklearn.preprocessing import MinMaxScaler
```

```
minmax = MinMaxScaler()  
data_tf = minmax.fit_transform(df)
```

```
df = pd.DataFrame(data_tf, columns=['x1', 'x2', 'x3'])
df.plot.kde()
```

<Axes: ylabel='Density'>



4. Robust Scaler

- Suited for data with outliers
- Calculate by subtracting 1st-quartile & div by difference between 3rd-quartile & 1st-quartile

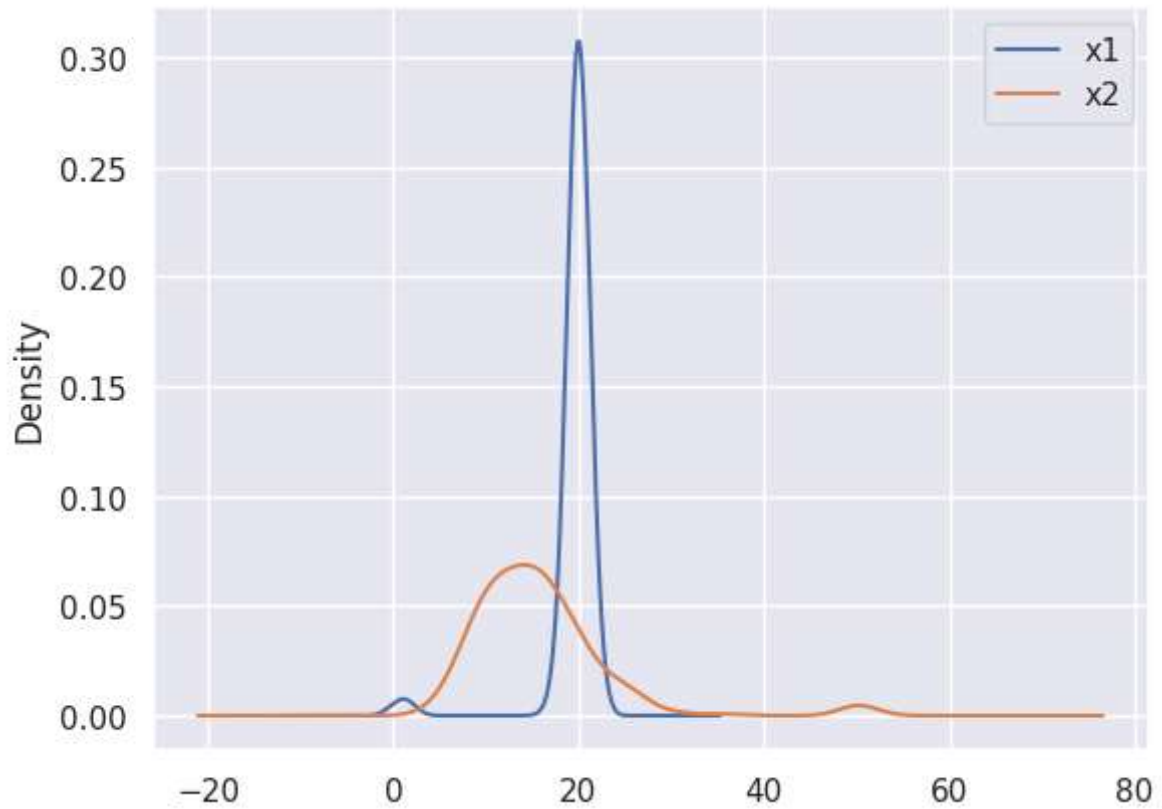
$$\frac{x_i - Q_1(x)}{Q_3(x) - Q_1(x)}$$

```
df = pd.DataFrame({
    # Distribution with lower outliers
    'x1': np.concatenate([np.random.normal(20, 1, 1000), np.random.normal(1, 1, 25)]),
    # Distribution with higher outliers
    'x2': np.concatenate([np.random.chisquare(15, 1000), np.random.normal(50, 1, 25)]),
})
```



```
df.plot.kde()
```

<Axes: ylabel='Density'>



```
from sklearn.preprocessing import RobustScaler
robustscaler = RobustScaler()
```

```
data_tf = robustscaler.fit_transform(df)
df = pd.DataFrame(data_tf, columns=['x1', 'x2'])
```

df

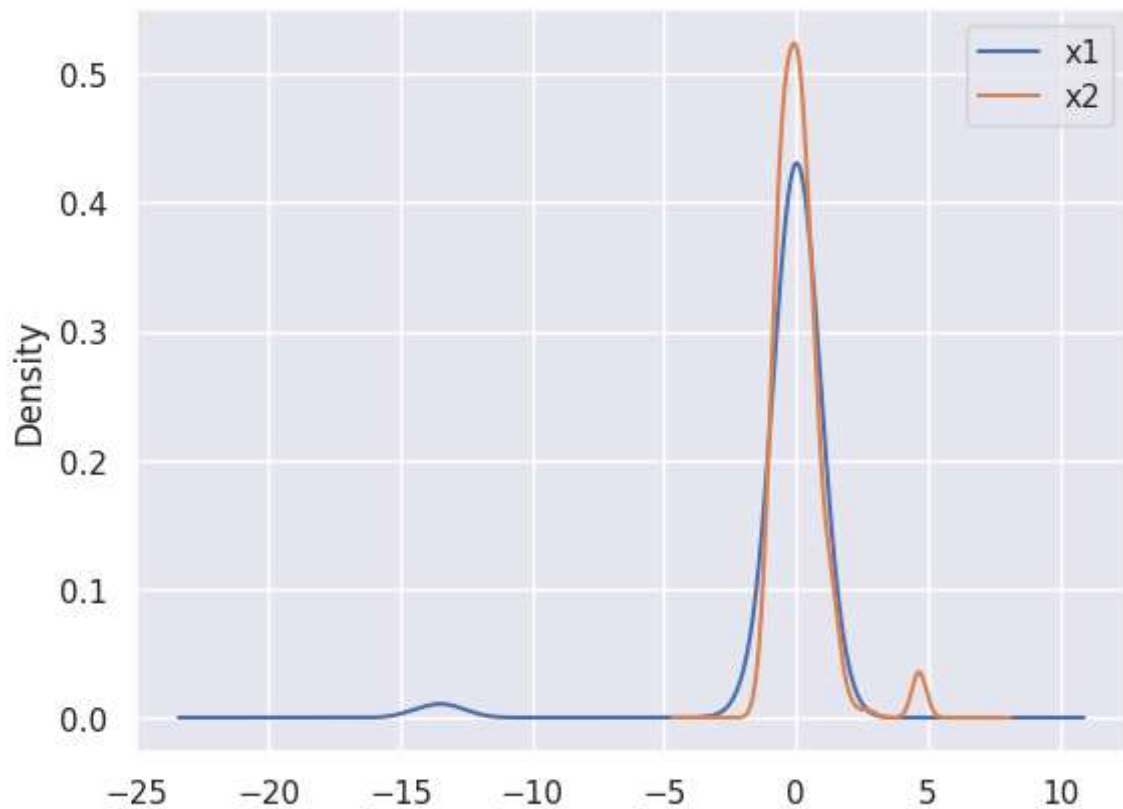
	x1	x2
0	1.074620	0.969802
1	1.169918	-1.108542
2	-0.542149	0.139105
3	-0.442237	-0.656917
4	0.166445	0.536792
...	...	...
1020	-13.563157	4.514469
1021	-13.570401	4.792702
1022	-13.343961	4.756797
1023	-13.516608	4.727187
1024	-13.068275	4.640128

```
df.describe()
```

	x1	x2
count	1025.000000	1025.000000
mean	-0.314049	0.133781
std	2.211162	0.991235
min	-14.825119	-1.496884
25%	-0.487669	-0.490086
50%	0.000000	0.000000
75%	0.512331	0.509914
max	2.318372	4.905623

```
df.plot.kde()
```

<Axes: ylabel='Density'>



5. Normalizer

- Each parameter value is obtained by dividing by magnitude

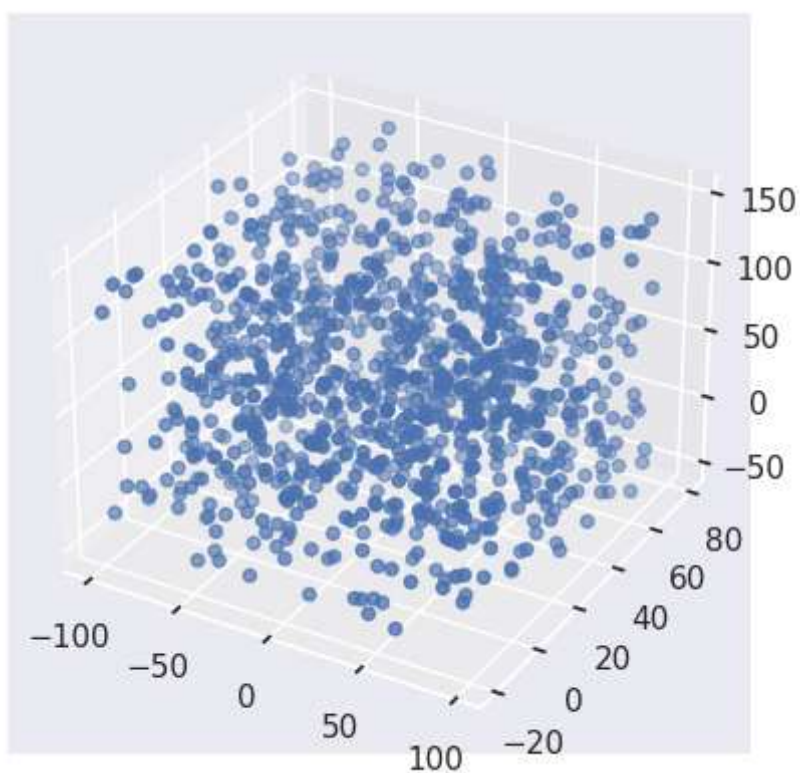
$$\frac{x_i}{\sqrt{x_i^2 + y_i^2 + z_i^2}}$$

- Centralizes data to origin

```
df = pd.DataFrame({  
    'x1': np.random.randint(-100, 100, 1000).astype(float),  
    'y1': np.random.randint(-20, 80, 1000).astype(float),  
    'z1': np.random.randint(-50, 150, 1000).astype(float),  
})
```

```
fig = plt.figure()  
ax = plt.axes(projection='3d')  
ax.scatter3D(df.x1, df.y1, df.z1)
```

<mpl_toolkits.mplot3d.art3d.Path3DCollection at 0x7ce8c061a2a0>



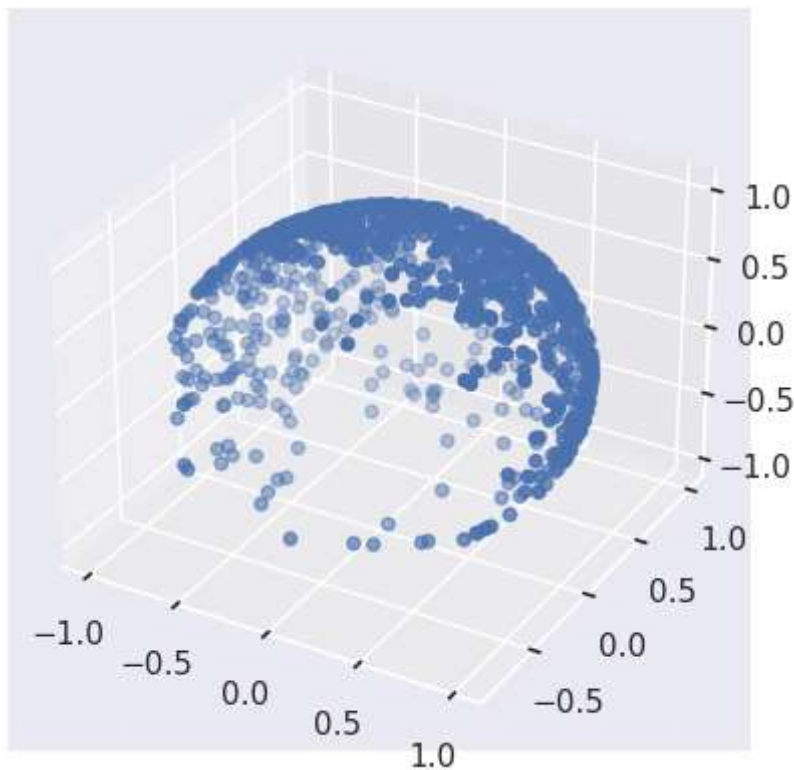
```
from sklearn.preprocessing import Normalizer
```

```
normalizer = Normalizer()  
data_tf = normalizer.fit_transform(df)
```

```
df = pd.DataFrame(data_tf, columns=['x1', 'y1', 'z1'])
```

```
ax = plt.axes(projection='3d')
ax.scatter3D(df.x1, df.y1, df.z1)
```

```
<mpl_toolkits.mplot3d.art3d.Path3DCollection at 0x7ce8c05409e0>
```



6. Binarization

- Thresholding numerical values to binary values (0 or 1)
- A few learning algorithms assume data to be in Bernoulli distribution - Bernoulli's Naive Bayes

```
X = np.array([[ 1., -1.,  2.],
               [ 2.,  0.,  0.],
               [ 0.,  1., -1.]])
```

```
from sklearn.preprocessing import Binarizer
binarizer = Binarizer(threshold=1.0)
```

```
data_tf = binarizer.fit_transform(X)
```

```
data_tf
```

```
array([[0., 0., 1.],  
       [1., 0., 0.],  
       [0., 0., 0.]])
```

7. Encoding Categorical Values

Encoding Ordinal Values

- Ordinal Values - Low, Medium & High. Relationship between values
- LabelEncoding with right mapping

```
df = pd.DataFrame({  
    'Age': [33, 44, 22, 44, 55, 22],  
    'Income': ['Low', 'Low', 'High', 'Medium', 'Medium', 'High']})
```

```
df
```

	Age	Income
0	33	Low
1	44	Low
2	22	High
3	44	Medium
4	55	Medium
5	22	High

```
df.Income.map({'Low':1, 'Medium':2, 'High':3})
```

	Income
0	1
1	1
2	3
3	2
4	2
5	3

PS: We can use transformer class for this as well, we will see that later

Encoding Nominal Values

- Nominal Values - Male, Female. No relationship between data
- One Hot Encoding for converting data into one-hot vector

```
df = pd.DataFrame({
    'Age': [33, 44, 22, 44, 55, 22, 35],
    'Gender': ['Male', 'Female', 'Male', 'Female', 'Male', 'Male', 'LGBTQ']})
```

```
df.Gender.unique()
```

```
array(['Male', 'Female', 'LGBTQ'], dtype=object)
```

```
print(pd.get_dummies(df.Gender))
```

```

      Female  LGBTQ  Male
0    False  False  True
1     True  False  False
2    False  False  True
3     True  False  False
4    False  False  True
5    False  False  True
6    False   True  False

```

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
```

```
le = LabelEncoder()
df['gender_tf'] = le.fit_transform(df.Gender)
```

```
df
```

	Age	Gender	gender_tf
0	33	Male	2
1	44	Female	0
2	22	Male	2
3	44	Female	0
4	55	Male	2
5	22	Male	2
6	35	LGBTQ	1

```
OneHotEncoder().fit_transform(df[['gender_tf']]).toarray()
```

```
array([[0., 0., 1.],
       [1., 0., 0.],
       [0., 0., 1.],
       [1., 0., 0.],
       [0., 0., 1.],
       [0., 0., 1.],
       [0., 1., 0.]])
```

```
df = pd.DataFrame({
    'Age': [33, 44, 22, 44, 55, 22],
    'Income': ['Low', 'Low', 'High', 'Medium', 'Medium', 'High']})
```

```
df['Income_label'] = le.fit_transform(df.Income)
```

df

	Age	Income	Income_label
0	33	Low	1
1	44	Low	1
2	22	High	0
3	44	Medium	2
4	55	Medium	2
5	22	High	0

8. Imputation

- Missing values cannot be processed by learning algorithms
- Imputers can be used to infer value of missing data from existing data

```
df = pd.DataFrame({
    'A': [1, 2, 3, 4, np.nan, 7],
    'B': [3, 4, 10, np.nan, 4, 5]
})
```

```
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='mean')
```

```
df_copy = imputer.fit_transform(df)
```

```
df_copy
```

```
array([[ 1. ,  3. ],
       [ 2. ,  4. ],
       [ 3. , 10. ],
       [ 4. ,  5.2],
       [ 3.4,  4. ],
       [ 7. ,  5. ]])
```

9. Polynomial Features

- Deriving non-linear feature by converting data into higher degree
- Used with linear regression to learn model of higher degree

```
df = pd.DataFrame({'A':[1,2,3,4,5], 'B':[2,3,4,5,6]})
df
```

	A	B
0	1	2
1	2	3
2	3	4
3	4	5
4	5	6

```
from sklearn.preprocessing import PolynomialFeatures
pol = PolynomialFeatures(degree=2, include_bias=True)
```

```
pol.fit_transform(df)
```

```
array([[ 1.,  1.,  2.,  1.,  2.,  4.],
       [ 1.,  2.,  3.,  4.,  6.,  9.],
       [ 1.,  3.,  4.,  9., 12., 16.],
       [ 1.,  4.,  5., 16., 20., 25.],
       [ 1.,  5.,  6., 25., 30., 36.]])
```

10. Custom Transformer

- Often, you will want to convert an existing Python function into a transformer to assist in data cleaning or processing.
- FunctionTransformer is used to create one Transformer
- `validate = False`, is required for string columns


```
from sklearn.preprocessing import FunctionTransformer
```

```
def mapping(x):  
    x['Age'] = x['Age']+2  
    x['Counter'] = x['Counter'] * 2  
    return x
```

```
customtransformer = FunctionTransformer(mapping, validate=False)
```

```
df = pd.DataFrame({  
    'Age': [33,44,22,44,55,22],  
    'Counter': [3,4,2,4,5,2],  
})  
df
```

	Age	Counter
0	33	3
1	44	4
2	22	2
3	44	4
4	55	5
5	22	2

```
customtransformer.transform(df)
```

	Age	Counter
0	35	6
1	46	8
2	24	4
3	46	8
4	57	10
5	24	4

```
print (type(customtransformer))
```

```
<class 'sklearn.preprocessing._function_transformer.FunctionTransformer'>
```

11. HashingVectorizer

- All above techniques converts data into table where each word is converted to column
- Learning on data with lakhs of columns is difficult to process

- HashingVectorizer is an useful technique for out-of-core learning
- Multiple words are hashed to limited column
- Limitation - Hashed value to word mapping is not possible

```
df = pd.DataFrame({
    'A': [1, 2, 3, 4, np.nan, 7],
    'B': [3, 4, 1, np.nan, 4, 5],
    'Text': ['I', 'am', 'going', 'the', 'opposite', 'way']
})
```

```
from sklearn.feature_extraction.text import HashingVectorizer
hv = HashingVectorizer(n_features=5)
```

```
hv.fit_transform(df.Text).toarray()
```

```
array([[ 0.,  0.,  0.,  0.,  0.],
       [ 1.,  0.,  0.,  0.,  0.],
       [ 0.,  0.,  0.,  0., -1.],
       [ 0.,  0.,  0., -1.,  0.],
       [ 0., -1.,  0.,  0.,  0.],
       [ 0.,  0.,  0.,  0.,  1.]])
```

12. Image Processing using skimage

- skimage doesn't come with anaconda. install with 'pip install skimage'
- Images should be converted from 0-255 scale to 0-1 scale.
- skimage takes image path & returns numpy array
- images consist of 3 dimension

```
! pip install scikit-image
```

```
Requirement already satisfied: scikit-image in /usr/local/lib/python3.12/dist-packages
(0.25.2)
Requirement already satisfied: numpy≥1.24 in /usr/local/lib/python3.12/dist-packages (from
scikit-image) (2.0.2)
Requirement already satisfied: scipy≥1.11.4 in /usr/local/lib/python3.12/dist-packages
(from scikit-image) (1.16.3)
Requirement already satisfied: networkx≥3.0 in /usr/local/lib/python3.12/dist-packages
(from scikit-image) (3.5)
Requirement already satisfied: pillow≥10.1 in /usr/local/lib/python3.12/dist-packages
(from scikit-image) (11.3.0)
Requirement already satisfied: imageio≠2.35.0, ≥2.33 in /usr/local/lib/python3.12/dist-
packages (from scikit-image) (2.37.2)
Requirement already satisfied: tifffile≥2022.8.12 in /usr/local/lib/python3.12/dist-
packages (from scikit-image) (2025.10.16)
Requirement already satisfied: packaging≥21 in /usr/local/lib/python3.12/dist-packages
(from scikit-image) (25.0)
Requirement already satisfied: lazy-loader≥0.4 in /usr/local/lib/python3.12/dist-packages
(from scikit-image) (0.4)
```

```
from skimage.io import imread, imshow
```

```
image = imread('/content/dog.jpg')
```

```
image.shape
```

```
(430, 710, 3)
```

image

ndarray (430, 710, 3) [show data](#)



```
array([[56, 64, 17],
       [56, 64, 17],
       [55, 63, 16],
       ...,
       [43, 35, 14],
       [43, 35, 14],
       [43, 35, 14]],

      [[55, 63, 16],
       [55, 63, 16],
       [55, 63, 16],
       ...,
       [43, 35, 14],
       [43, 35, 14],
       [43, 35, 14]],

      [[54, 62, 15],
       [54, 62, 15],
       [54, 62, 15],
       ...,
       [43, 35, 14],
       [43, 35, 14],
       [43, 35, 14]],

      ...,

      [[64, 76, 12],
       [64, 76, 12],
```

```
[65, 77, 13],
    ...,
[79, 89, 37],
[79, 89, 37],
[78, 88, 36]],

[[63, 75, 11],
 [64, 76, 12],
 [64, 76, 12],
    ...,
 [79, 89, 37],
 [78, 88, 36],
 [78, 88, 36]],

[[63, 75, 11],
 [63, 75, 11],
 [64, 76, 12],
    ...,
 [78, 88, 36],
 [78, 88, 36],
 [77, 87, 35]]], dtype=uint8)
```

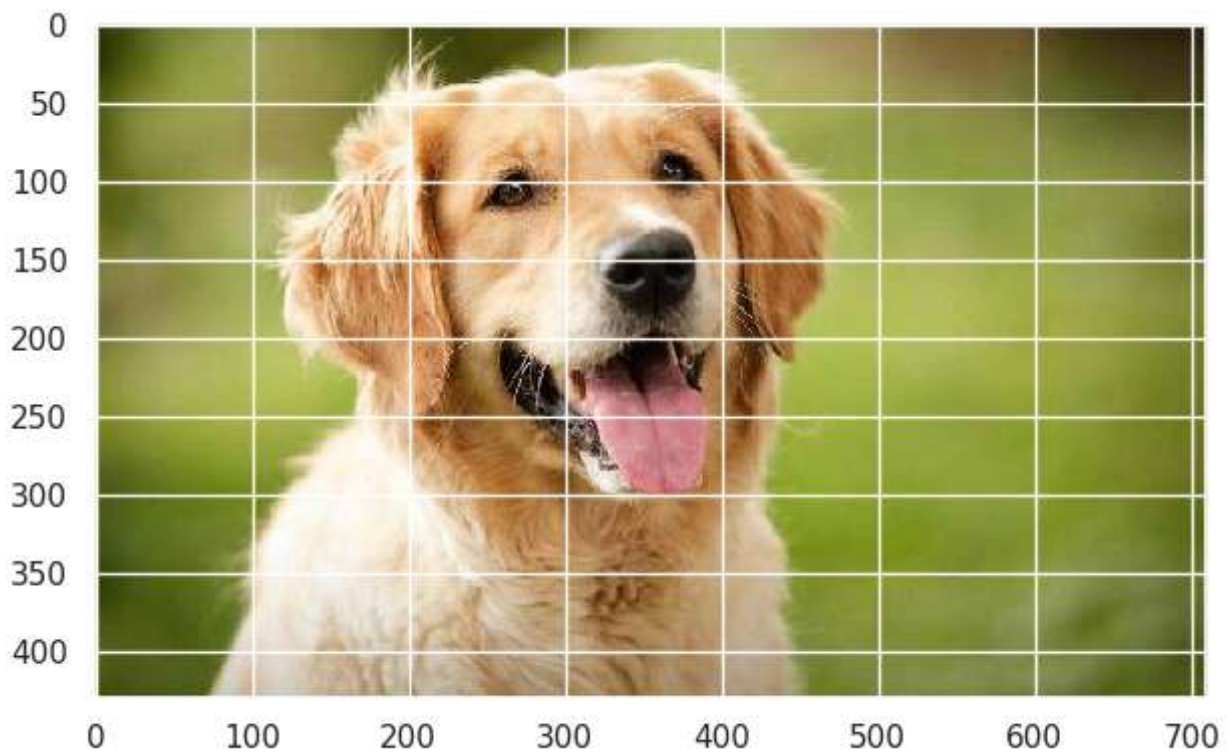
image[0]

```
array([[56, 64, 17],
       [56, 64, 17],
       [55, 63, 16],
       ...,
       [43, 35, 14],
       [43, 35, 14],
       [43, 35, 14]], dtype=uint8)
```

```
imshow(image)
```

```
/tmp/ipython-input-3355601266.py:1: FutureWarning: `imshow` is deprecated since version
0.25 and will be removed in version 0.27. Please use `matplotlib`, `napari`, etc. to
visualize images.
    imshow(image)
```

```
<matplotlib.image.AxesImage at 0x7ce8be180260>
```



```
from skimage.color import rgb2gray
```

```
rgb2gray(image).shape
```

```
(430, 710)
```

```
print(rgb2gray(image))
```

```
[[0.23102471 0.23102471 0.22710314 ... 0.13798392 0.13798392 0.13798392]
 [0.22710314 0.22710314 0.22710314 ... 0.13798392 0.13798392 0.13798392]
 [0.22318157 0.22318157 0.22318157 ... 0.13798392 0.13798392 0.13798392]
 ...
 [0.26994353 0.26994353 0.2738651 ... 0.32598353 0.32598353 0.32206196]
 [0.26602196 0.26994353 0.26994353 ... 0.32598353 0.32206196 0.32206196]
 [0.26602196 0.26602196 0.26994353 ... 0.32206196 0.32206196 0.31814039]]
```

```
rgb_image = rgb2gray(image)
```

```
rgb_image
```

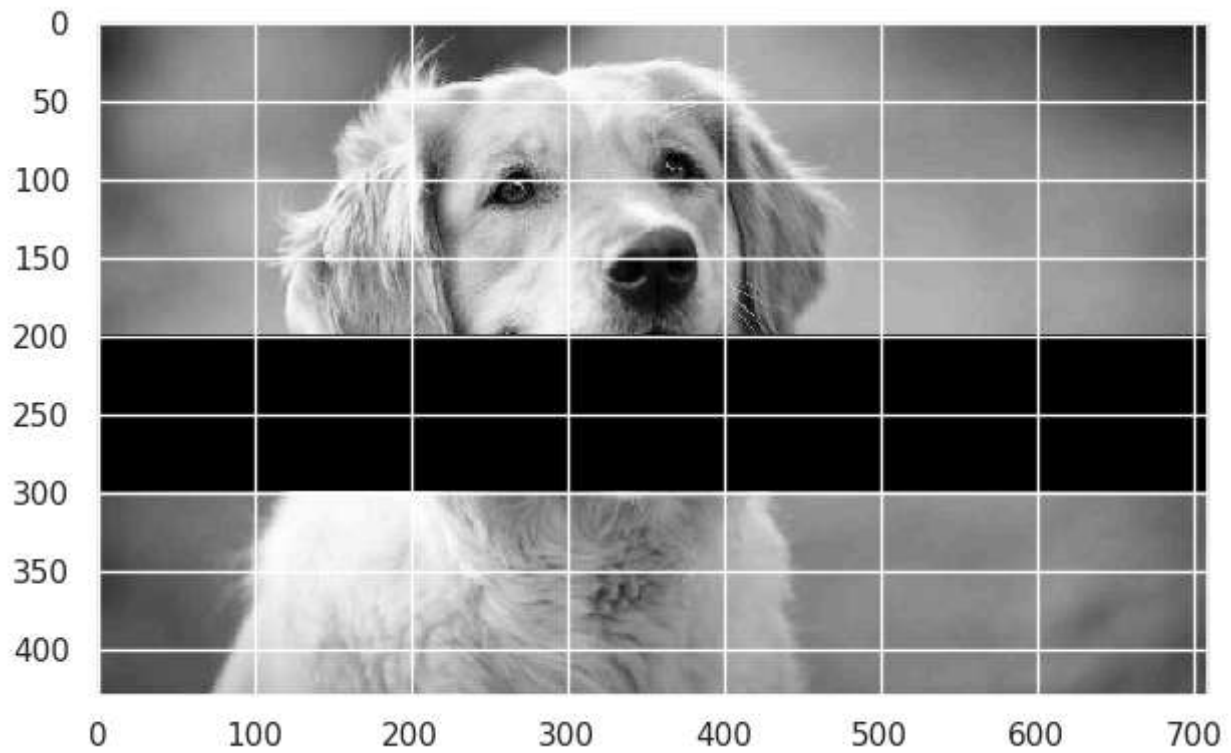
```
array([[0.23102471, 0.23102471, 0.22710314, ..., 0.13798392, 0.13798392,
        0.13798392],
       [0.22710314, 0.22710314, 0.22710314, ..., 0.13798392, 0.13798392,
        0.13798392],
       [0.22318157, 0.22318157, 0.22318157, ..., 0.13798392, 0.13798392,
        0.13798392],
       ...,
       [0.26994353, 0.26994353, 0.2738651 , ..., 0.32598353, 0.32598353,
        0.32206196],
       [0.26602196, 0.26994353, 0.26994353, ..., 0.32598353, 0.32206196,
        0.32206196],
       [0.26602196, 0.26602196, 0.26994353, ..., 0.32206196, 0.32206196,
        0.31814039]])
```

```
rgb_image[200:300]= 0
```

```
imshow(rgb_image)
```

```
/tmp/ipython-input-349728104.py:1: FutureWarning: `imshow` is deprecated since version 0.25
and will be removed in version 0.27. Please use `matplotlib`, `napari`, etc. to visualize
images.
imshow(rgb_image)
```

```
<matplotlib.image.AxesImage at 0x7ce8be597140>
```



```
from skimage.transform import resize
```



```
imshow(resize(image, (150,200)))
```

```
/tmp/ipython-input-4182925553.py:1: FutureWarning: `imshow` is deprecated since version  
0.25 and will be removed in version 0.27. Please use `matplotlib`, `napari`, etc. to  
visualize images.  
imshow(resize(image, (150,200)))
```

```
<matplotlib.image.AxesImage at 0x7ce8be59f980>
```

